

Software Engineering

Detailed Semester Exam Notes | Theory, Diagrams, Tables and Concepts

Prepared for Rahul Yadav. These notes cover software process models, requirements, design, UML, testing, maintenance, project management, estimation, risk and quality assurance according to the provided syllabus.

Unit	Syllabus Area	Exam Focus
I	Software product and process	Models, merits/demerits, CMM, metrics
II	Requirement elicitation, analysis and SRS	Functional/non-functional requirements, use cases, validation, traceability
III	Software design	Design principles, UML, architecture, UI, SA/SD, metrics
IV	Software analysis and testing	Static/dynamic analysis, BVA, EP, white-box, cyclomatic complexity
V	Maintenance and project management	SCM, estimation, COCOMO, scheduling, risk, SQA

Exam tip: for process model questions, always write definition, diagram, steps, merits and demerits. For testing questions, include test cases. For project management, include formulas and estimation steps.

Rahul yadav

UNIT I: Software Product and Software Process

Software is a collection of programs, data, documentation and operating procedures that solve a user problem. A software product is not only code; it also includes manuals, configuration files, test data and maintenance support.

Software engineering is a disciplined approach to development, operation and maintenance of software. It applies engineering principles to produce reliable, maintainable, cost-effective and high-quality software.

A software process is a framework of activities used to build software. Common activities include communication, planning, modeling, construction, testing, deployment and maintenance.

- Software is developed, not manufactured in the traditional sense.
- Software does not wear out, but it deteriorates due to changes and poor maintenance.
- Good software should be correct, reliable, usable, efficient, secure and maintainable.
- Process gives discipline and repeatability to development work.



Term	Meaning
Software Product	Complete deliverable used by customer
Software Process	Set of activities to develop software
Software Engineering	Engineering approach for quality software
Product Metric	Measures product attributes
Process Metric	Measures development process performance

Software Product and Process Characteristics

Software product characteristics describe quality of the final software. These include functionality, reliability, usability, efficiency, maintainability, portability, security and scalability.

Software process characteristics describe quality of the method used to build software. A good process is understandable, visible, supportable, acceptable, reliable, robust and maintainable.

A strong process improves predictability, reduces project risk and helps teams deliver software within time and budget.

- Functionality means software provides required features.
- Reliability means software performs correctly for specified time.
- Usability means users can learn and operate it easily.
- Maintainability means software can be modified with less effort.
- Process visibility helps managers track progress.

Product Characteristic	Meaning
Correctness	Meets specified requirements
Reliability	Works without failure
Efficiency	Uses resources effectively
Usability	Easy to learn and operate
Maintainability	Easy to modify and fix
Portability	Can run in different environments

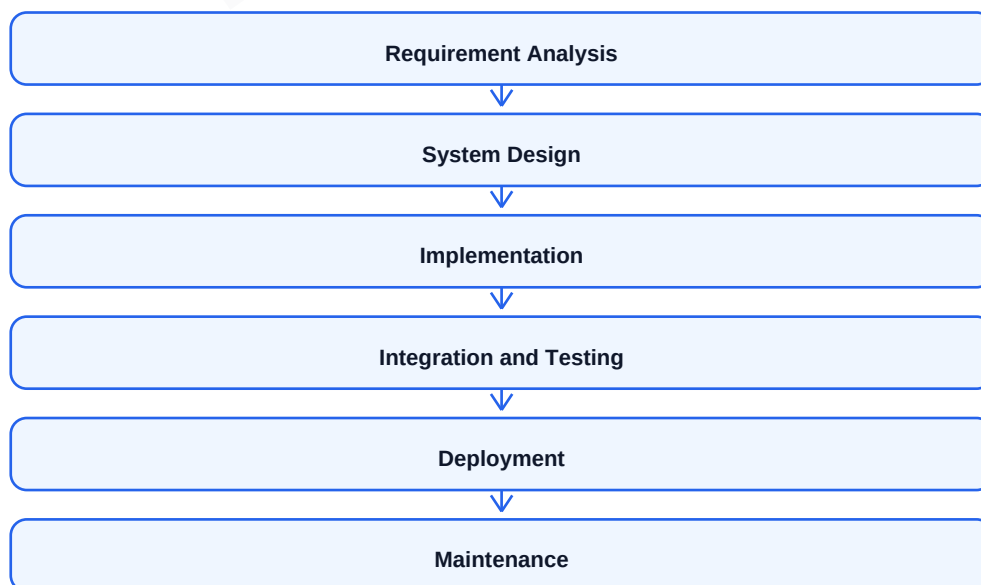
Linear Sequential Model / Waterfall Model

The waterfall model is a linear sequential software development model. It divides development into ordered phases: requirements, design, implementation, testing, deployment and maintenance. Each phase should be completed before the next phase begins.

It is simple and easy to manage because phases and deliverables are clearly defined. It is suitable for small projects where requirements are clear and stable.

The main limitation is poor flexibility. If requirements change late, revisiting earlier phases becomes costly.

- Best suited for stable and well-understood requirements.
- Documentation is strong because every phase has deliverables.
- Testing starts late, so defects may be discovered late.
- Customer feedback is limited during development.
- It is not ideal for uncertain or rapidly changing projects.



Merits	Demerits
Simple and easy to understand	Difficult to handle changing requirements
Clear milestones and documentation	Working software available late
Good for small stable projects	High risk if early requirements are wrong

Merits	Demerits
Easy project tracking	Testing starts after implementation

Rahul yadav

Prototyping Model and RAD Model

The prototyping model creates an early working model of the software to understand user requirements. The prototype may be throwaway or evolutionary. Users evaluate the prototype and provide feedback.

Rapid Application Development focuses on fast development using reusable components, tools, user involvement and time-boxed delivery. It is useful when business requirements are urgent and modular.

Both models improve user feedback compared with waterfall. However, poor planning may lead to weak architecture or unrealistic user expectations.

- Prototype clarifies unclear requirements.
- Throwaway prototype is discarded after learning requirements.
- Evolutionary prototype gradually becomes final system.
- RAD needs skilled team and strong user involvement.
- RAD works well when system can be divided into modules.

Model	Main Idea	Best Use
Prototyping	Build quick model and refine requirements	Unclear user interface/requirements
RAD	Fast development using components/tools	Time-critical business applications
Waterfall	Sequential phases	Stable requirements
Incremental	Deliver in parts	Prioritized features

Evolutionary Models: Incremental and Spiral

The incremental model develops software in small increments. Each increment delivers a working subset of the final product. High-priority functions are delivered early and later increments add more features.

The spiral model combines iterative development with risk analysis. Each loop of the spiral includes planning, risk analysis, engineering and customer evaluation. It is suitable for large, complex and high-risk projects.

Evolutionary models are useful when requirements evolve over time. They allow feedback and reduce risk by delivering software gradually.

- Incremental model provides early partial working software.
- Spiral model explicitly focuses on risk management.
- Customer feedback is available after each iteration.
- Management can prioritize important features.
- Spiral model can be costly due to risk analysis effort.



Model	Strength	Weakness
Incremental	Early delivery and feedback	Needs good architecture planning
Spiral	Strong risk handling	Costly and complex
Prototype	Clarifies requirements	May become poorly structured
RAD	Fast delivery	Needs skilled team and tools

Component Assembly, RUP and Agile Processes

Component assembly model builds software by integrating reusable components. Components may be existing modules, libraries, frameworks or services. This reduces development time and improves reliability when components are already tested.

RUP, or Rational Unified Process, is an iterative and use-case driven process. It has four phases: inception, elaboration, construction and transition. It emphasizes architecture, risk and disciplined development.

Agile process is iterative, incremental and customer-focused. It values working software, collaboration, adaptability and frequent delivery.

- Component model needs proper component selection and integration.
- RUP is architecture-centric and risk-driven.
- Agile accepts changing requirements.
- Scrum and XP are popular agile methods.
- Agile needs active customer participation and self-organizing teams.

Process	Main Feature	Suitable For
Component Assembly	Reuse existing components	Systems with reusable modules
RUP	Iterative, architecture-centric	Large disciplined projects
Agile	Short iterations and feedback	Changing requirements
Scrum	Sprints and product backlog	Team-based product development

Process Customization, Improvement, CMM and Metrics

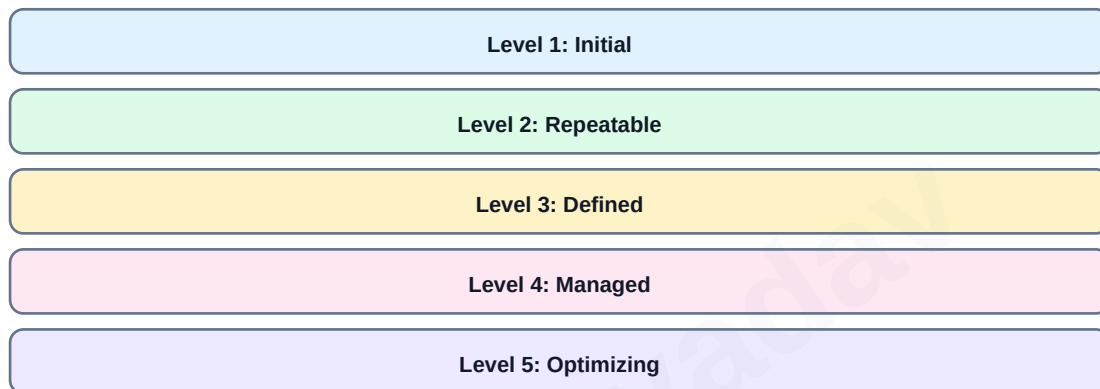
Software process customization means adapting a standard process to fit project size, risk, team, technology and customer needs. No single process model is best for every project.

Process improvement focuses on making the development process more predictable and effective. CMM, or Capability Maturity Model, describes maturity levels of an organization's software process.

Metrics are quantitative measures. Product metrics measure software attributes such as size, complexity and defects. Process metrics measure development performance such as productivity, defect removal efficiency and schedule variance.

- CMM Level 1: Initial, ad-hoc process.
- CMM Level 2: Repeatable project management.
- CMM Level 3: Defined organization process.
- CMM Level 4: Managed using metrics.
- CMM Level 5: Optimizing through continuous improvement.

CMM Maturity Levels



Metric Type	Examples
Product Metrics	LOC, function points, complexity, defect density
Process Metrics	Productivity, review effectiveness, defect removal efficiency
Project Metrics	Effort, cost, schedule, staffing, risk count
Quality Metrics	Reliability, maintainability, customer-reported defects

UNIT II: Requirement Elicitation and Requirements

Requirement engineering is the process of discovering, analyzing, documenting, validating and managing requirements. A requirement describes what the system should do or what quality/constraint it should satisfy.

Functional requirements describe services, features and behavior of the system. Non-functional requirements describe quality attributes and constraints such as performance, security, usability and reliability.

Good requirements should be correct, complete, unambiguous, consistent, verifiable, traceable and feasible.

- Functional requirement example: user shall login using email and password.
- Non-functional requirement example: system shall respond within 2 seconds.
- Domain requirements come from application area rules.
- Requirement errors are costly if found late.
- SRS is the official requirement document.

Requirement Type	Meaning	Example
Functional	What system should do	Generate monthly report
Non-functional	Quality/constraint	Response time under 2 sec
Domain	Business/domain rule	GST calculation rule
Interface	External interaction	Payment gateway API
Constraint	Restriction on solution	Use MySQL database

Requirement Sources and Elicitation Techniques

Requirement sources include customers, end users, domain experts, existing systems, business documents, standards, laws, operational environment and competitors. Different sources may have conflicting expectations.

Elicitation means gathering requirements from stakeholders. Common techniques include interviews, questionnaires, observation, document analysis, brainstorming, workshops, prototyping and use case analysis.

The analyst must ask clear questions, listen carefully, resolve conflicts and document assumptions. Elicitation is iterative because requirements become clearer over time.

- Interview gives detailed information but takes time.
- Questionnaire collects information from many users quickly.
- Observation reveals real work practices.
- Prototyping helps users visualize requirements.
- Workshops help resolve conflicts among stakeholders.

Technique	Advantage	Limitation
Interview	Deep understanding	Time consuming
Questionnaire	Large audience	Limited detail
Observation	Shows actual workflow	Observer effect
Workshop	Consensus building	Needs facilitation
Prototype	Clear user feedback	May create false final-product expectation

Analysis Modeling: Function-Oriented and Object-Oriented

Analysis modeling represents requirements in a structured form before design. It helps understand data, functions, behavior and objects of the system.

Function-oriented analysis focuses on processes and data flow. Data Flow Diagrams show how data moves through processes, data stores and external entities. Structured analysis is useful for transaction-processing systems.

Object-oriented analysis focuses on objects/classes, their attributes, operations and relationships. It is closer to modern object-oriented programming and is commonly modeled using UML.

- Function-oriented view asks: what functions transform input into output?
- Object-oriented view asks: what objects exist and how do they collaborate?
- DFD is common in structured analysis.
- Class diagram and use case diagram are common in OO analysis.
- Both methods help remove ambiguity from requirements.

Approach	Focus	Main Models
Function-Oriented	Functions and data flow	DFD, data dictionary, process specification
Object-Oriented	Objects and interactions	Use case, class, sequence diagrams
Structured	Top-down decomposition	ERD, DFD
OO	Encapsulation and relationships	UML diagrams

Use Case Modeling

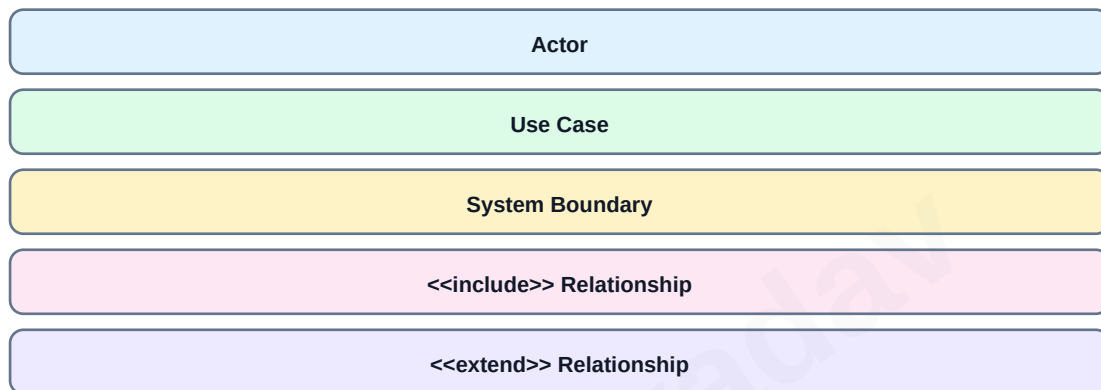
Use case modeling describes system functionality from user's point of view. It identifies actors, use cases and relationships between them. An actor is an external entity that interacts with the system.

A use case represents a goal-oriented interaction such as Login, Place Order or Generate Report. Use case diagrams are useful for communicating requirements with customers because they are simple and visual.

Use case descriptions include preconditions, main flow, alternate flow, exceptions and postconditions.

- Actor may be human user, external system or device.
- Use case name should be action-oriented.
- include relationship represents common mandatory behavior.
- extend relationship represents optional or conditional behavior.
- Use cases help derive test cases.

Use Case Model Elements



Element	Meaning
Actor	External role interacting with system
Use Case	Service/goal provided by system
System Boundary	Scope of system
Include	Reusable mandatory sub-use-case
Extend	Optional behavior under condition

SRS, Requirement Validation and Traceability

Software Requirements Specification is a complete document describing functional and non-functional requirements, interfaces, constraints, assumptions and acceptance criteria. It acts as agreement between customer and developer.

Requirement validation checks whether requirements are correct, complete, consistent, realistic and testable. Validation techniques include reviews, prototyping, test-case generation and consistency checking.

Requirement traceability means linking requirements to their sources, design elements, code modules and test cases. It helps impact analysis when requirements change.

- SRS should be unambiguous and verifiable.
- Validation prevents building the wrong product.
- Traceability matrix maps requirements to design/code/test.
- Forward traceability follows requirement to implementation.
- Backward traceability follows implementation back to requirement.

SRS Quality	Meaning
Correct	Represents actual customer need
Complete	All requirements included
Consistent	No contradictions
Verifiable	Can be tested or checked
Traceable	Can be linked to source and implementation
Modifiable	Easy to update

UNIT III: Software Design Process and Principles

Software design transforms requirements into a blueprint for implementation. It defines architecture, components, interfaces, data structures, algorithms and user interface.

Design bridges the gap between requirement analysis and coding. A good design reduces complexity, improves maintainability and supports testing.

Important design concepts include abstraction, modularity, information hiding, functional independence, refinement, architecture, patterns and separation of concerns.

- Abstraction focuses on essential details and hides unnecessary complexity.
- Modularity divides system into manageable modules.
- Information hiding hides internal implementation details.
- Refinement gradually adds detail to design.
- Functional independence means high cohesion and low coupling.



Principle	Meaning
Modularity	Divide system into modules
Abstraction	Show essential features
Information Hiding	Hide internal details
Low Coupling	Reduce dependency between modules
High Cohesion	Keep related responsibilities together

Software Modeling and UML

Software modeling creates simplified representations of the system. Models help stakeholders understand structure, behavior and interactions before implementation.

UML, or Unified Modeling Language, is a standard visual modeling language for object-oriented systems. It includes structural diagrams and behavioral diagrams.

Common UML diagrams include use case diagram, class diagram, sequence diagram, activity diagram, state diagram, component diagram and deployment diagram.

- Class diagram shows classes, attributes, operations and relationships.
- Sequence diagram shows messages over time.
- Activity diagram shows workflow.
- State diagram shows object states and transitions.
- Component diagram shows software components and dependencies.

UML Diagram	Purpose
Use Case	Functional requirements from user view
Class	Static object structure
Sequence	Object interaction over time
Activity	Workflow/business process
State	State changes of object
Deployment	Hardware/software deployment view

Architectural Design, Views and Styles

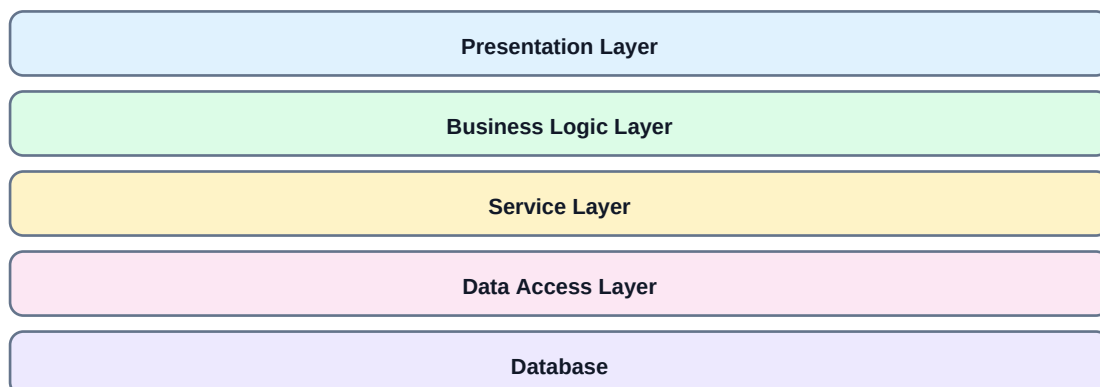
Architectural design defines the high-level structure of a software system. It identifies major components, their responsibilities, relationships and communication mechanisms.

Architectural views describe the system from different stakeholder perspectives. Common views include logical view, process view, development view, physical/deployment view and use case view.

Architectural styles are reusable organization patterns such as layered architecture, client-server, pipe-and-filter, repository, microkernel and MVC.

- Layered style separates system into layers like UI, business and data.
- Client-server divides service provider and service requester.
- Pipe-and-filter is useful for data transformation pipelines.
- Repository style uses shared central data store.
- MVC separates model, view and controller.

Layered Architecture



Style	Main Idea	Example
Layered	Separate levels of responsibility	Web app layers
Client-Server	Clients request services	Banking app
Pipe-Filter	Data passes through filters	Compiler phases
Repository	Central data store	Version control

Style	Main Idea	Example
MVC	Separate UI, logic and control	Spring MVC

Rahul yadav

User Interface Design

User interface design focuses on how users interact with software. A good interface is easy to learn, efficient, consistent, responsive and error tolerant.

UI design should consider user profile, tasks, environment, accessibility and feedback. Poor UI can make technically correct software difficult to use.

Important principles include consistency, visibility, simplicity, feedback, error prevention, user control and accessibility.

- Use consistent terminology, colors and layout.
- Provide meaningful error messages.
- Minimize user memory load.
- Support undo/confirmation for risky actions.
- Design forms with clear labels and validation.

Principle	Meaning
Consistency	Same actions and patterns behave same
Feedback	System shows result of user action
Simplicity	Avoid unnecessary complexity
Error Prevention	Prevent invalid input where possible
Accessibility	Usable by people with different abilities

Function-Oriented, SA/SD and Component-Based Design

Function-oriented design decomposes the system into functions or modules. It is often based on structured analysis and Data Flow Diagrams. Structure charts may be used to represent module hierarchy.

SA/SD stands for Structured Analysis and Structured Design. It uses DFDs, data dictionary, process specifications and structure charts to move from requirements to modular design.

Component-based design builds software using reusable components with well-defined interfaces. It improves productivity and maintainability when components are reliable and compatible.

- Function-oriented design emphasizes transformation of input to output.
- SA/SD is top-down and systematic.
- Component-based design supports reuse.
- Components should have clear interfaces.
- Loose coupling between components improves replaceability.

Design Type	Focus	Main Artifact
Function-Oriented	Functions and data flow	Structure chart
Object-Oriented	Objects/classes	Class diagram
Component-Based	Reusable components	Component diagram
Architectural	High-level system structure	Architecture diagram

Design Metrics

Design metrics measure quality of design before coding or during development. They help identify complex, tightly coupled or poorly structured parts of system.

Common design metrics include coupling, cohesion, fan-in, fan-out, depth of inheritance tree, number of children, response for class and cyclomatic complexity for control flow.

A good design usually has high cohesion, low coupling, understandable interfaces and limited complexity.

- Coupling measures dependency between modules.
- Cohesion measures relatedness of responsibilities inside a module.
- Fan-in counts modules that call a module.
- Fan-out counts modules called by a module.
- Complexity metrics help estimate testing and maintenance effort.

Metric	Meaning	Preferred Direction
Coupling	Interdependence between modules	Low
Cohesion	Relatedness inside module	High
Fan-in	Number of callers	Can be high for utility modules
Fan-out	Number of called modules	Controlled
Cyclomatic Complexity	Number of independent paths	Low/moderate

UNIT IV: Software Analysis and Testing Fundamentals

Software testing is the process of executing or evaluating software to find defects and verify that it meets requirements. Testing improves confidence but cannot prove absence of all defects.

Verification checks whether the product is built correctly according to specifications. Validation checks whether the correct product is built according to user needs.

Static analysis examines software artifacts without executing program. Dynamic analysis involves executing program and observing behavior.

- Testing detects defects, it does not guarantee defect-free software.
- Verification asks: are we building the product right?
- Validation asks: are we building the right product?
- Static testing includes reviews and inspections.
- Dynamic testing includes unit, integration, system and acceptance testing.

Concept	Meaning
Error	Human mistake
Fault/Defect	Bug in software artifact
Failure	Incorrect behavior during execution
Test Case	Input, execution condition and expected result
Test Oracle	Source of expected result

Static Analysis, Dynamic Analysis and Code Inspections

Static analysis reviews code, design or documents without running the program. It can detect defects early, such as coding standard violations, unreachable code, security weaknesses and requirement inconsistencies.

Dynamic analysis executes the program with test cases and checks actual output. It is necessary to detect runtime errors, performance problems and behavior under real inputs.

Code inspection is a formal review process where reviewers examine code to find defects. It usually includes planning, overview, preparation, inspection meeting, rework and follow-up.

- Static analysis is cheaper because defects are found early.
- Dynamic analysis confirms actual runtime behavior.
- Inspection is more formal than walkthrough.
- Review checklists improve defect detection.
- Automated static tools can detect style and security issues.

Technique	Execution Required?	Examples
Static Analysis	No	Reviews, inspections, linting
Dynamic Analysis	Yes	Unit tests, system tests
Inspection	No	Formal peer review
Walkthrough	No	Author-led review
Testing	Yes	Run test cases

Software Testing Process, Levels and Criteria

The testing process includes test planning, test design, test environment setup, test execution, defect reporting, regression testing and test closure. A test plan defines scope, strategy, resources, schedule, risks and deliverables.

Testing levels include unit testing, integration testing, system testing and acceptance testing. Unit testing checks individual modules. Integration testing checks module interactions. System testing checks complete system. Acceptance testing checks customer acceptance.

Test criteria decide when testing is adequate. Examples include requirement coverage, branch coverage, path coverage, defect density and risk coverage.

- Unit testing is often done by developers.
- Integration testing can be top-down, bottom-up or sandwich.
- System testing validates complete system behavior.
- Acceptance testing is performed by customer/users.
- Regression testing checks that changes did not break existing features.



Level	Focus
Unit Testing	Individual function/class/module
Integration Testing	Interfaces and interactions
System Testing	Complete integrated system
Acceptance Testing	Business/user acceptance
Regression Testing	Existing functionality after changes

Rahul yadav

Black-Box Testing: Equivalence Partitioning and BVA

Black-box testing tests software based on specifications without knowing internal code. The tester focuses on input and output behavior.

Equivalence partitioning divides input domain into classes where each class is expected to behave similarly. One representative value from each valid and invalid class is selected.

Boundary Value Analysis tests values at the boundaries of input ranges because defects often occur near limits. For range 1 to 100, test 0, 1, 2, 99, 100 and 101.

- EP reduces number of test cases while maintaining coverage.
- BVA focuses on min, min+1, nominal, max-1, max and just outside boundaries.
- Black-box testing is useful for functional validation.
- It does not require knowledge of source code.
- Decision table testing is useful for business rules.

Technique	Idea	Example
Equivalence Partitioning	Divide input into valid/invalid classes	Age 18-60 valid, below/above invalid
Boundary Value Analysis	Test around boundaries	17,18,19,59,60,61
Decision Table	Combinations of conditions/actions	Loan approval rules
State Transition	State-based behavior	Login lock after 3 attempts

White-Box Testing and Cyclomatic Complexity

White-box testing tests internal structure, logic and paths of the program. The tester uses knowledge of code to design test cases. It includes statement coverage, branch coverage, condition coverage and path coverage.

Cyclomatic complexity measures number of linearly independent paths in a program's control flow graph. It helps estimate minimum number of test cases needed for basis path testing.

Formula: $V(G) = E - N + 2P$, where E is number of edges, N is number of nodes and P is number of connected components. For one connected program graph, P is usually 1.

- Cyclomatic complexity can also be computed as number of decision nodes + 1.
- Higher complexity means more paths and more testing effort.
- Basis path testing designs test cases for independent paths.
- White-box testing is usually done at unit level.
- Very high complexity indicates design should be simplified.

Coverage	Meaning
Statement Coverage	Every statement executes at least once
Branch Coverage	Every decision outcome executes
Condition Coverage	Each boolean condition true and false
Path Coverage	Independent execution paths covered
Basis Path	Uses cyclomatic complexity

Cyclomatic Complexity:

$$V(G) = E - N + 2P$$

For single connected graph: $V(G) = E - N + 2$

Alternative: $V(G) = \text{number of decision nodes} + 1$

Test Case Design, Test Oracles, Metrics and Tools

Test case design is the process of selecting inputs, execution conditions and expected outputs. A good test case has objective, precondition, input data, steps, expected result and actual result.

A test oracle is a mechanism used to determine whether test output is correct. It can be requirement document, previous version, mathematical formula, expert judgment or automated assertion.

Test metrics help evaluate testing progress and quality. Tools support test management, automation, defect tracking, coverage measurement and performance testing.

- Defect density = number of defects / size of software.
- Test coverage measures how much requirement/code is tested.
- Defect removal efficiency measures defects removed before release.
- Automation is useful for repeated regression tests.
- Manual exploratory testing is useful for usability and unexpected scenarios.

Metric/Tool Area	Examples
Test Metrics	Coverage, pass rate, defect density, DRE
Test Management	Test plans and cases
Automation Tools	Selenium, JUnit, TestNG
Defect Tracking	Jira, Bugzilla
Performance Testing	JMeter, LoadRunner

Object-Oriented Analysis/Design and Comparison with Structured SE

Object-oriented analysis and design models a system as interacting objects. It supports encapsulation, inheritance, polymorphism and abstraction. OO analysis identifies classes, objects, relationships and responsibilities.

Structured software engineering decomposes system into functions and data flow. Object-oriented software engineering decomposes system into objects that combine data and behavior.

OO approach is suitable for systems where real-world entities and interactions are important. Structured approach is simple for process-oriented systems.

- OO approach improves reuse through classes and inheritance.
- Structured approach uses DFD and structure charts.
- OO design uses UML diagrams.
- Structured design separates data and functions.
- OO design combines data and methods inside objects.

Basis	Structured Approach	Object-Oriented Approach
Decomposition	Functions/processes	Objects/classes
Main Models	DFD, structure chart	UML diagrams
Data and Behavior	Separate	Encapsulated together
Reuse	Function/module reuse	Class/component reuse
Change Handling	Can affect many functions	Often localized in classes

UNIT V: Software Maintenance

Software maintenance is the process of modifying software after delivery to correct faults, improve performance, adapt to changed environment or enhance functionality. Maintenance often consumes a large part of total software cost.

Types of maintenance are corrective, adaptive, perfective and preventive. Corrective fixes faults. Adaptive modifies software for changed environment. Perfective improves functionality or performance. Preventive improves maintainability and prevents future problems.

Good documentation, modular design and configuration management reduce maintenance difficulty.

- Corrective maintenance fixes discovered defects.
- Adaptive maintenance handles new OS, hardware, laws or interfaces.
- Perfective maintenance adds/improves features.
- Preventive maintenance refactors or improves structure.
- Maintainability should be considered during design itself.

Type	Purpose	Example
Corrective	Fix faults	Correct wrong tax calculation
Adaptive	Adapt to environment	Support new database
Perfective	Improve features/performance	Add dashboard
Preventive	Prevent future issues	Refactor complex module

SCM, Change Management, Version Control and Reporting

Software Configuration Management controls changes in software artifacts such as source code, documents, test cases, builds and releases. It ensures integrity and traceability of product versions.

Version control records changes over time and allows teams to collaborate safely. Change control evaluates requested changes, approves or rejects them and tracks implementation.

Change reporting provides visibility into what changed, who changed it, why it changed and which version contains the change.

- Configuration item is any artifact placed under control.
- Baseline is an approved snapshot of configuration items.
- Version control tools include Git and SVN.
- Change Control Board approves important changes.
- SCM prevents confusion caused by uncontrolled file versions.



SCM Activity	Meaning
Identification	Identify configuration items
Version Control	Manage artifact versions
Change Control	Approve and control changes
Status Accounting	Report change status
Audit	Verify correctness of configuration

Program Comprehension, Re-engineering and Reverse Engineering

Program comprehension is the process of understanding existing software before modifying it. It involves reading code, documentation, architecture, data flow and dependencies.

Reverse engineering analyzes existing software to recover design, architecture or requirements. It moves from code to higher-level abstraction. It does not necessarily change the software.

Re-engineering modifies or restructures existing software to improve maintainability, performance or adaptability. It may include code restructuring, data restructuring and documentation improvement.

- Program comprehension is difficult when documentation is poor.
- Reverse engineering helps understand legacy systems.
- Re-engineering improves existing system without replacing it completely.
- Refactoring is a small-scale restructuring technique.
- Tool support includes code analyzers, dependency visualizers and documentation generators.

Term	Meaning
Program Comprehension	Understanding existing code and behavior
Reverse Engineering	Recover higher-level information from code
Re-engineering	Analyze and transform existing system
Forward Engineering	Build implementation from design
Refactoring	Improve internal structure without changing behavior

Project Management Concepts and Feasibility Analysis

Software project management plans, organizes, monitors and controls software development. It includes scope, time, cost, quality, risk, communication, resources and stakeholders.

Feasibility analysis checks whether a proposed project is practical and worthwhile. It includes technical feasibility, economic feasibility, operational feasibility, schedule feasibility and legal feasibility.

Project planning defines objectives, deliverables, tasks, resources, schedule, cost, quality goals and risks.

- Technical feasibility checks whether required technology and skills are available.
- Economic feasibility compares cost and benefit.
- Operational feasibility checks whether users can and will use system.
- Schedule feasibility checks whether deadline is realistic.
- Legal feasibility checks compliance with laws and contracts.

Project Management Areas



Feasibility Type	Question
Technical	Can we build it with available technology?
Economic	Is benefit greater than cost?
Operational	Will it work in user environment?
Schedule	Can it be completed on time?
Legal	Is it legally acceptable?

Effort, Schedule and Cost Estimation with COCOMO

Estimation predicts effort, time, cost and resources required for a software project. Accurate estimation helps planning, budgeting and staffing. Common inputs are size, complexity, team capability and project constraints.

COCOMO, or Constructive Cost Model, estimates effort based on software size measured in KLOC. Basic COCOMO classifies projects as organic, semi-detached and embedded.

Basic COCOMO formulas are: Effort = $a(KLOC)^b$ person-months and Development Time = $c(Effort)^d$ months. Coefficients depend on project type.

- Organic projects are small/simple with experienced teams.
- Semi-detached projects are medium complexity with mixed experience.
- Embedded projects are complex with tight constraints.
- Cost can be estimated from effort and cost per person-month.
- COCOMO II extends original model for modern development.

Project Type	Effort Formula	Time Formula
Organic	$E = 2.4(KLOC)^{1.05}$	$T = 2.5(E)^{0.38}$
Semi-detached	$E = 3.0(KLOC)^{1.12}$	$T = 2.5(E)^{0.35}$
Embedded	$E = 3.6(KLOC)^{1.20}$	$T = 2.5(E)^{0.32}$

Example:

If KLOC = 10 and project is organic:

Effort = $2.4 * (10)^{1.05}$ person-months

Development Time = $2.5 * (Effort)^{0.38}$ months

Resource Allocation, Scheduling and Tracking

Resource allocation assigns people, tools, hardware, software and budget to project tasks. The goal is to use resources efficiently without overloading team members.

Project scheduling breaks project into tasks, estimates duration, identifies dependencies and creates timeline. Common tools include Gantt chart, PERT chart and CPM network.

Project tracking compares actual progress with planned progress. Managers monitor milestones, effort spent, defects, risks and schedule variance.

- Gantt chart shows tasks against calendar time.
- PERT handles uncertain activity durations.
- CPM identifies critical path that determines minimum project duration.
- Milestones mark important review or delivery points.
- Tracking helps detect delay early.

Technique	Use
Work Breakdown Structure	Divide project into manageable tasks
Gantt Chart	Schedule visualization
PERT	Probabilistic time estimation
CPM	Critical path identification
Milestone Tracking	Monitor important checkpoints

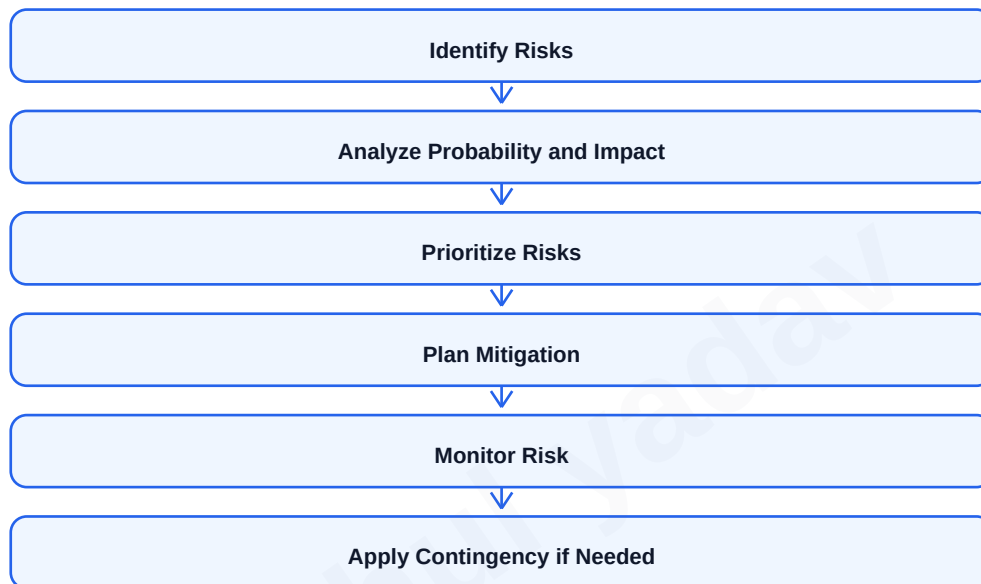
Risk Assessment, Mitigation and Project Plan

Risk is an uncertain event that can negatively affect project objectives. Risk management includes identification, analysis, prioritization, mitigation, monitoring and contingency planning.

Risk assessment evaluates probability and impact of each risk. Risk exposure can be calculated as probability multiplied by loss. High exposure risks should be handled first.

A project plan documents scope, schedule, cost, resources, quality plan, risk plan, communication plan, configuration management plan and monitoring approach.

- Common risks include requirement changes, staff turnover, technology failure, schedule pressure and cost overrun.
- Risk mitigation reduces probability or impact.
- Contingency plan defines what to do if risk occurs.
- Risk monitoring tracks risk status throughout project.
- Project plan acts as roadmap for execution and control.



Risk	Mitigation
Requirement Change	Change control and prototyping
Staff Turnover	Documentation and knowledge sharing
Schedule Delay	Buffer time and tracking
Technology Risk	Proof of concept
Quality Risk	Reviews and testing strategy

Software Quality Assurance and Project Metrics

Software Quality Assurance is a planned and systematic set of activities that ensures software process and product conform to requirements, standards and procedures.

SQA includes process definition, reviews, audits, testing support, standards enforcement, defect analysis, quality reporting and continuous improvement. It prevents defects instead of only detecting them.

Project metrics help managers evaluate progress and quality. Examples include effort variance, schedule variance, defect density, productivity, review effectiveness and customer satisfaction.

- Quality assurance is process-oriented; quality control is product-oriented.
- Reviews and audits are important SQA activities.
- Standards improve consistency.
- Metrics support objective management decisions.
- Continuous improvement uses lessons learned from projects.

Metric	Formula/Meaning
Defect Density	Defects / software size
Productivity	Output size / effort
Schedule Variance	Planned time - actual time or percentage variance
Effort Variance	Planned effort vs actual effort
DRE	Defects removed before release / total defects

Important Exam Questions and Quick Revision

Important long-answer topics are waterfall model, prototyping, RAD, spiral model, agile process, CMM, functional and non-functional requirements, SRS, use case modeling, UML, architectural styles, testing levels, black-box and white-box testing, cyclomatic complexity, COCOMO and risk management.

For numerical questions, practice cyclomatic complexity and COCOMO. For testing questions, practice boundary value and equivalence partitioning test cases. For requirement questions, practice writing functional and non-functional requirements from a small problem statement.

For design questions, draw simple diagrams and mention high cohesion, low coupling, modularity and information hiding.

- Waterfall answer: diagram + phases + merits/demerits.
- Requirement answer: definition + types + SRS qualities.
- Testing answer: levels + techniques + test cases.
- Cyclomatic complexity: $V(G)=E-N+2P$ or decision nodes + 1.
- COCOMO: effort and development time formulas.
- Risk answer: identify, analyze, mitigate, monitor.
- SQA answer: standards, reviews, audits, metrics and improvement.

Topic	Must Write in Exam
Software Process Models	Definition, diagram, advantages, disadvantages
SRS	Characteristics and structure
Use Case	Actor, use case, include/extend
UML	Diagram types and purpose
Testing	Black-box, white-box, levels
Maintenance	Types and examples
COCOMO	Project type + formulas